Lesson: Overlay UI Components

Hello everyone! Welcome back to our second lesson on Android development with Jetpack Compose.

In our first session, we covered a *lot* of ground. We learned how to set up a screen with `Column` and `Row`, get user input with `TextField`, and even navigate between screens and save data with `Room` and `ViewModel`. You've built the solid foundation and "skeleton" of an app.

Today, we're going to focus on making your apps feel polished and interactive. In Python, you might use `print()` for feedback or `input()` to ask a question. In a mobile app, those actions are disruptive. Instead, we use **overlay UI components**—elements that appear *on top* of your main screen to provide context, feedback, or ask for decisions.

We'll cover four main components:

- 1. **Menus** (in the App Bar)
- 2. **Dialogs** (for important decisions)
- 3. **Snackbars** (for quick feedback)
- 4. **Bottom Sheets** (for contextual options)

Let's dive in.

1. Menus (in the `TopAppBar`)

This is the most common menu you'll see. It's the "three dots" (or "kebab") menu in the corner of an app. We add this using the `actions` parameter of the `TopAppBar` (which you've already learned about).

The magic here is combining an `IconButton` (for the dots) with a `DropdownMenu`. The key concept is **state**: we need to *remember* if the menu is open or closed.

```
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.MoreVert
import androidx.compose.material3.*
import androidx.compose.runtime.*

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MyAppBarWithMenu() {
    // 1. State to remember if the menu is expanded or not
    var menuExpanded by remember { mutableStateOf(false) }

    TopAppBar(
        title = { Text("My App") },
        actions = {
            // 2. Icon button to toggle the menu
            IconButton(onClick = { menuExpanded = true }) {
                Icon(Icons.Default.MoreVert, contentDescription = "More c
            }

            // 3. The menu itself
            DropdownMenu(
                expanded = menuExpanded,
                onDismissRequest = { menuExpanded = false } // Close if u
            ) {
                DropdownMenuItem(
                    text = { Text("Settings") },
                    onClick = {
                        // Handle settings click
                        menuExpanded = false // Close the menu
                    }
                )
                DropdownMenuItem(
                    text = { Text("Profile") },
                    onClick = {
                        // Handle profile click
                        menuExpanded = false // Close the menu
                    }
                )
            }
        }
    )
}
```

Key Takeaway: You control the `DropdownMenu` 's visibility with a `Boolean` state (`menuExpanded`). The `IconButton` sets it to `true` , and `onDismissRequest` (clicking

outside) or clicking an item sets it back to `false`.

2. Dialogs (`AlertDialog`)

A dialog is a small window that **blocks the UI** to get an immediate user decision. Use this for critical actions, like confirming a deletion.

Like menus, a dialog is also controlled by a `Boolean` state.

```
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.*

@Composable
fun MyDeleteConfirmationDialog(
    showDialog: Boolean,
    onDismiss: () -> Unit,
    onConfirm: () -> Unit
) {
    if (showDialog) {
        AlertDialog(
            onDismissRequest = onDismiss, // User clicked outside or back
            title = { Text("Confirm Deletion") },
            text = { Text("Are you sure you want to delete this item? Thi

            // Confirm button
            confirmButton = {
                TextButton(
                    onClick = {
                        onConfirm()
                        onDismiss() // Close dialog after confirming
                    }
                ) {
                    Text("Delete")
                }
            },

            // Dismiss button (optional)
            dismissButton = {
                TextButton(onClick = onDismiss) {
```

```
                    Text("Cancel")
                }
            }
        )
    }
}

// --- How you would USE this ---
@Composable
fun MyScreen() {
    var showDialog by remember { mutableStateOf(false) }

    Column {
        Button(onClick = { showDialog = true }) {
            Text("Delete Item")
        }
    }

    MyDeleteConfirmationDialog(
        showDialog = showDialog,
        onDismiss = { showDialog = false },
        onConfirm = { /* TODO: Call your ViewModel to delete the item */
    )
}
```

3. Snackbars (Quick Feedback)

A `Snackbar` is a **non-blocking** message that slides up from the bottom. It's perfect for simple feedback like "Message sent" or "Item saved."

Snackbars are a bit different. They don't use a simple `Boolean` state. They are managed by a `SnackbarHostState` and are launched using a **Coroutine**, which you've already been introduced to. They are almost always used inside a `Scaffold`.

```
import androidx.compose.material3.*
import androidx.compose.runtime.*
import kotlinx.coroutines.launch

@OptIn(ExperimentalMaterial3Api::class)
@Composable
```

```
fun MyScreenWithSnackbar() {
    // 1. Remember the SnackbarHostState
    val snackbarHostState = remember { SnackbarHostState() }
    // 2. Remember a CoroutineScope to launch the snackbar
    val scope = rememberCoroutineScope()

    Scaffold(
        // 3. Assign the state to the Scaffold's snackbar host
        snackbarHost = { SnackbarHost(snackbarHostState) },
        topBar = { TopAppBar({ Text("Snackbars") }) }
    ) { paddingValues ->

        Column(modifier = Modifier.padding(paddingValues)) {
            Button(
                onClick = {
                    // 4. Launch a coroutine to show the snackbar
                    scope.launch {
                        snackbarHostState.showSnackbar(
                            message = "Your item has been saved!",
                            actionLabel = "Undo", // Optional action
                            duration = SnackbarDuration.Short
                        )
                        // Result will be SnackbarResult.ActionPerformed
                    }
                }
            ) {
                Text("Save Item")
            }
        }
    }
}
```

4. Modal Bottom Sheets

A `ModalBottomSheet` slides up from the bottom and is used when you have more options than a simple menu but don't want to navigate to a whole new screen (e.g., a "Share" panel).

This component also uses a special state (`SheetState`) and a `CoroutineScope` to control it.

```
import androidx.compose.material3.*
import androidx.compose.runtime.*
import kotlinx.coroutines.launch

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MyScreenWithBottomSheet() {
    // 1. Remember the sheet state (initially hidden)
    val sheetState = rememberModalBottomSheetState()
    val scope = rememberCoroutineScope()
    var showBottomSheet by remember { mutableStateOf(false) }

    Scaffold { paddingValues ->
        Column(modifier = Modifier.padding(paddingValues)) {
            Button(onClick = { showBottomSheet = true }) {
                Text("Show Options")
            }
        }

        // 2. This is the Bottom Sheet composable
        if (showBottomSheet) {
            ModalBottomSheet(
                onDismissRequest = { showBottomSheet = false },
                sheetState = sheetState
            ) {
                // 3. This is the CONTENT of the sheet
                Column(modifier = Modifier.padding(16.dp)) {
                    Text("Share to...", style = MaterialTheme.typography.
                    // You would have Rows with Icons and Text here
                    Text("...")
                    Text("...")
                    Button(
                        onClick = {
                            scope.launch { sheetState.hide() }.invokeOnCo
                                if (!sheetState.isVisible) {
                                    showBottomSheet = false
                                }
                            }
                        }
                    ) {
                        Text("Close")
                    }
                }
            }
        }
```

```
    }
}
```

5. 💡 Examples: Dropdowns and Context Menus

You asked for specific examples of dropdowns with more features, and how to trigger them from list items.

A. Form Dropdown Menu (`ExposedDropdownMenuBox`)

For a form, you don't just want a menu, you want a "selector" or "picker." The best component for this is `ExposedDropdownMenuBox`.

```
import androidx.compose.material3.*
import androidx.compose.runtime.*

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MyFormDropdown() {
    val options = listOf("Kotlin", "Python", "Java", "Swift")
    var expanded by remember { mutableStateOf(false) }
    var selectedOptionText by remember { mutableStateOf(options[0]) }

    ExposedDropdownMenuBox(
        expanded = expanded,
        onExpandedChange = { expanded = !expanded }
    ) {
        // This TextField is what the user sees
        TextField(
            modifier = Modifier.menuAnchor(), // Important modifier
            readOnly = true,
            value = selectedOptionText,
            onValueChange = {},
            label = { Text("Language") },
            trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(exp
            colors = ExposedDropdownMenuDefaults.textFieldColors()
        )
```

```
        // This is the actual menu that appears
        ExposedDropdownMenu(
            expanded = expanded,
            onDismissRequest = { expanded = false }
        ) {
            options.forEach { selectionOption ->
                DropdownMenuItem(
                    text = { Text(selectionOption) },
                    onClick = {
                        selectedOptionText = selectionOption
                        expanded = false
                    }
                )
            }
        }
    }
}
```

B. Dropdown with Icons and Dividers

You can customize `DropdownMenuItem` easily. Just place `Divider` composables between them.

```
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Edit
import androidx.compose.material.icons.filled.Email
import androidx.compose.material.icons.filled.Share
import androidx.compose.material3.*
import androidx.compose.runtime.*

@Composable
fun MyAdvancedDropdown() {
    var expanded by remember { mutableStateOf(false) }

    Box { // We need a Box to anchor the menu
        Button(onClick = { expanded = true }) { Text("Show Menu") }

        DropdownMenu(
            expanded = expanded,
            onDismissRequest = { expanded = false }
        ) {
            DropdownMenuItem(
                text = { Text("Share") },
```

```
                onClick = { /* Handle share */ expanded = false },
                leadingIcon = {
                    Icon(Icons.Default.Share, contentDescription = "Share
                }
            )
            DropdownMenuItem(
                text = { Text("Edit") },
                onClick = { /* Handle edit */ expanded = false },
                leadingIcon = {
                    Icon(Icons.Default.Edit, contentDescription = "Edit")
                }
            )

            // --- Here is the divider ---
            Divider(modifier = Modifier.padding(vertical = 4.dp))

            DropdownMenuItem(
                text = { Text("Contact") },
                onClick = { /* Handle contact */ expanded = false },
                leadingIcon = {
                    Icon(Icons.Default.Email, contentDescription = "Conta
                }
            )
        }
    }
}
```

C. Triggering an Action on Long Press

You mentioned wanting to trigger actions on a "Long press on list items." This is a classic pattern for showing a context menu or a delete dialog.

We can use the `combinedClickable` modifier on our `LazyColumn` items.

```
import androidx.compose.foundation.combinedClickable
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.*
import androidx.compose.runtime.*

// (Requires the MyDeleteConfirmationDialog from section 2)

@OptIn(ExperimentalFoundationApi::class)
@Composable
```

```
fun MyListWithLongPress() {
    // State for our dialog
    var showDeleteDialog by remember { mutableStateOf(false) }
    var selectedItem by remember { mutableStateOf(null) }

    val myItems = listOf("Item 1", "Item 2", "Item 3", "Item 4")

    LazyColumn {
        items(myItems) { item ->
            ListItem(
                headlineContent = { Text(item) },
                modifier = Modifier.combinedClickable(
                    onClick = {
                        // Handle regular click (e.g., navigate)
                        println("Clicked on $item")
                    },
                    onLongClick = {
                        // Handle long click!
                        println("Long-clicked on $item")
                        selectedItem = item
                        showDeleteDialog = true
                    }
                )
            )
            Divider()
        }
    }

    // When showDeleteDialog becomes true, this dialog will appear
    MyDeleteConfirmationDialog(
        showDialog = showDeleteDialog,
        onDismiss = { showDeleteDialog = false },
        onConfirm = {
            println("DELETING $selectedItem")
            // TODO: Call ViewModel to delete selectedItem
        }
    )
}
```

That's a wrap on overlay components! By mastering menus, dialogs, snackbars, and bottom sheets, you can create a much more responsive and professional-feeling application.

Let me know if you'd like to dive deeper into any of these examples.

> ***Student Question:*** *For snackbar and bottomsheet, explain why is there a need to use rememberCoroutineScope. For snackbar, explain why is there a need to use snackbarhoststate and the need to remembermodalbottomsheetstate.*

TA Follow-up: Coroutines and State Explained

That's an excellent and very sharp observation! It gets to the very core of how Jetpack Compose handles UI components that aren't just *on* or *off*, but that *animate* or *change over time*.

The short answer is: **animation**.

Both `Snackbar` and `ModalBottomSheet` don't just appear; they slide into view. This animation takes time (e.g., 300 milliseconds). The functions that *start* these animations need to be special.

Let's break down each of your questions.

1. Why `rememberCoroutineScope`? (The "How")

In Compose, any function that needs to "wait" for something to finish without freezing the app's screen (the UI thread) must be a **`suspend` function**.

- Showing a snackbar (`snackbarHostState.showSnackbar(...)`) is a `suspend` function because it needs to "wait" for the snackbar to finish showing, and potentially wait for the user to click "Undo."
- Hiding a bottom sheet (`sheetState.hide()`) is also a `suspend` function because it needs to "wait" for the slide-down animation to complete.

Here's the problem: You can *only* call a `suspend` function from *another* `suspend` function or from within a **`CoroutineScope`**.

Your `Button` 's `onClick = { ... }` block is a *normal* function, not a `suspend` one.

```
Button(onClick = {
```

```
    // ERROR! This is a normal lambda, not a coroutine.
    // We can't call a suspend function here directly.
    sheetState.hide()
})
```

This is where `rememberCoroutineScope` comes in.

- It gives you a **`CoroutineScope`** that is "aware" of your composable's lifecycle.
- You can use this scope to **`launch`** a new coroutine from your normal `onClick` handler. This is like creating a "new, short-lived task" to run your `suspend` function.
- `remember` ensures you use the *same* scope every time your composable redraws (recomposes), which is crucial for performance.

```
// 1. Get a scope tied to this composable's lifecycle
val scope = rememberCoroutineScope()

Button(onClick = {
    // 2. NOW we can launch a new coroutine
    scope.launch {
        // 3. And call our suspend function safely inside it
        sheetState.hide()
    }
})
```

Python Analogy: Think of `scope.launch { ... }` as being similar to creating a new task with `asyncio.create_task(...)` in Python. You're "firing off" an asynchronous job from a regular, synchronous piece of code.

2. Why `SnackbarHostState`? (The "Queue")

You're right, we could *almost* control a snackbar with a simple `Boolean`: `var showSnackbar by remember { mutableStateOf(false) }`.

But what happens if the user taps the "Save" button 5 times very quickly?

- With a simple `Boolean`, the snackbar might flicker, or only show once.
- What you *want* is for 5 "Item saved!" messages to show, one after the other. You want a **queue**.

The **`SnackbarHostState`** is a "manager" object that does exactly this. Its job is to:

1. **Manage the queue** of upcoming snackbar messages.
2. **Control the `showSnackbar` suspend function**, which adds a new message to that queue and waits for its turn.
3. **Hold the current snackbar's data** (the message, the action label like "Undo", the duration).
4. **Handle dismissal** (either by timer or by the user clicking the action).

You **`remember`** it so that this one single "manager" object survives recompositions. If you didn't, you'd create a new, empty queue every time the state changed, and your snackbars would disappear.

3. Why `rememberModalBottomSheetState`? (The "Physical State")

This one is similar to the snackbar but for a different reason. A bottom sheet has a much more complex "physical" state than just `true` or `false`.

A `ModalBottomSheetState` needs to track:

- Is it fully **hidden**?
- Is it fully **expanded**?
- Is it **half-expanded** (an optional in-between state)?
- Is it **currently animating** between any of these states?
- What is its **exact pixel offset** from the bottom of the screen right now?

A simple `Boolean` can't possibly hold all that information. The `ModalBottomSheetState` is the "controller" object that stores all this complex physical state.

And just like the `SnackbarHostState`, it also provides the `suspend` functions (`show()`, `hide()`) to *animate* between those states, which is why you need a `CoroutineScope` to call them.

In Short:

Component	State Object	Why you need it?
`CoroutineScope`	`rememberCoroutineScope()`	To **launch** animations (`suspend` functions) from normal `onClick` events.
`Snackbar`	`remember { SnackbarHostState() }`	To manage a **queue** of messages and their data (e.g., "Undo").
`BottomSheet`	`rememberModalBottomSheetState()`	To manage the complex **physical state** (hidden, expanded, animating, offset).

Great questions! This shows you're thinking about *why* the code is written the way it is, which is the most important skill for a developer.